How to use the two mock lab exams

Two Python-only practice papers in the style of the real final lab exam. Sit them under exam conditions, mark yourself with the marking scheme, then repeat the weak parts.

What is in each folder

File	Use
`MockA/MockA_Paper.md`	The paper (print this). Pandas + matplotlib, SQLite, PyTorch.
`MockA/cafe_orders.csv`	Data for Mock A.
`MockA/MockA_solution.py`	Model solution. Do NOT open until you have finished.
`MockA/MockA_marking_scheme_and_walkthrough.md`	Marks, traps, answers.
`MockB/MockB_Paper.md`	The paper (print this). Console logger, pandas/numpy, SQLite.
`MockB/module_scores.csv`, `MockB/stdin.txt`	Starter data and scripted keyboard input.
`MockB/MockB_solution.py`, `MockB/MockB_marking_scheme_and_walkthrough.md`	As above.

Mock A mirrors the 2024 paper (3 tasks, 40 marks). Mock B mirrors the "Practice Exam Task 3" style (no PyTorch). Do A first, then B a day later.

Setting up (5 minutes)

1. Make a fresh empty folder, e.g. `mock_A_attempt/`.
2. Copy ONLY the data file(s) (and `stdin.txt` for B) into it. Do not copy the solution or marking scheme.
3. Create `your_name_python_task.py` in that folder. Run it from that folder (`python your_name_python_task.py`), so relative file names work.
4. For Mock A you need `pandas`, `matplotlib` and `torch` installed; for Mock B only `pandas`, `numpy`, `matplotlib`.
5. Allowed: your printed notes, the topic documents, past papers. Not allowed: the solution file, the web, other people.

Time budget (the real mid-term test is 1 hour = 60 minutes)

Block	Mock A	Mock B
Read the whole paper	3	3
Task 1	20	18
Task 2	17	14
Task 3	14	17
Final run, comments, file name, "upload"	6	8

These two mocks were written longer than the real test; if you cannot finish in 60 minutes, that is normal - note which parts you left, and finish them untimed afterwards. (Your first timed attempt is for finding your pace.)

If a block is over time by 5 minutes, **stop, leave a comment `# TODO: not finished`, and move on**. Partial code earns partial marks; an unattempted task earns none.

Habits that earn marks: run the program after every sub-task; keep one `.py` file in the order of the paper; put the sub-task name as a comment above each part (`# Task 1a: load the data`); use clear variable names.

Marking yourself (30-40 minutes)

1. Run your file once from a clean folder (delete any `.db` file first). Does it run top to bottom without error? (A crash in Task 1 would lose everything after it, so this is the first thing a marker checks.)
2. Compare your printed numbers with section 2 of the marking scheme.
3. Go down the marks table. Award a mark only when you can point at the line of code that earns it. Half marks for "right idea, small bug".
4. Run it a second time without deleting anything: the database counts must not change (idempotence). For Mock B, also run with only `q`.
5. Read the "planted traps" table: tick the traps you handled, note the ones you missed.
6. Open the model solution only now. Do not just copy it: compare idea by idea and retype the part you got wrong from memory.
7. Answer the viva questions out loud in one or two sentences each.
8. Write your score and the three weakest sub-tasks on the first page.

Rough guide: 34+ strong; 26-33 good, fix the traps; below 26, redo the related topic documents first, then retry that task only.

If you get stuck during the mock

- **Error message:** read the LAST line; it names the problem. Look at the line number above it.
- **Stuck on one sub-task for 10 minutes:** write what you know as comments, make the rest of the file run, and move on.
- **Not sure of a cleaning rule:** pick a sensible one, write a comment saying why (e.g. `# blank item -> "Unknown"`). Markers reward a justified choice.
- **No plot window appears:** make sure `plt.show()` is called and that you are not running on a machine without a display.
- **SQLite "database is locked" / old rows:** close other programs using the file; delete the `.db` file and re-run.
- **Check your tools:** `print(df.head())`, `print(df.isna().sum())`, `print(df.dtypes)`, `SELECT * FROM table LIMIT 5` solve most mysteries.
- Never leave the exam with a file that does not run: comment out the broken part with `#` rather than submit a crash.

Mock C - practical-assessment style (added later)

Tests the instructor's official checklist exactly: pandas (load, shape, duplicates, merge, filter, save), matplotlib (scatter, line, bar, box), SQLite (to_sql, `?` queries, UPDATE, commit, export to CSV) and PyTorch regression (seed, `y = X @ w + b + noise`, Linear + ReLU, loss curve). 40 marks.

File	Use
`MockC/MockC_Paper.md`	The paper (print this).
`MockC/customers.csv`, `MockC/orders.csv`	The two data files (share `CustomerID`).
`MockC/MockC_solution.py`	Model solution; open only after you finish.
`MockC/MockC_marking_scheme_and_walkthrough.md`	Marks PER OPERATION, the checklist-to-topic-document table, traps, answers.

- **Setup:** as for A and B: a fresh folder, copy only the two CSV files, needs `pandas`, `matplotlib`, `torch`.
- **Time (60 min, like the real test):** read 3, Section 1 pandas 12, Section 2 plots 8, Section 3 SQLite 15, Section 4 PyTorch 14, final run and tidy 8.
- **Marking:** the scheme gives each small operation its own mark, so mark line by line. Then use its first table (checklist bullet -> topic document) to see which topic document to re-read for each lost mark. Do not memorise the solution; retype single operations from memory (one merge, one `?` query, one training loop).
- **Idempotence:** run twice; every number must be identical (the table is rebuilt each run).

- **If stuck:** the sections are independent except that 2-3 use the merged DataFrame from Section 1 and Section 4 stands alone - do Section 4 first if pandas is blocking you.